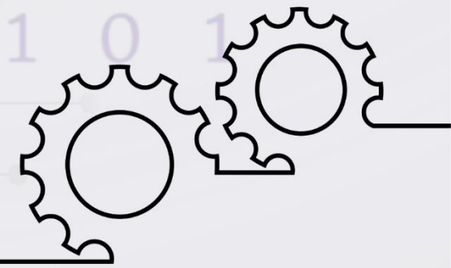
SIMATS
School of Engineering

Statistics with R Programming

Computer Science and Engineering



Saveetha Institute of Medical And Technical Sciences,Chennai.

INDEX

Module 1 Page No. 1
- *Basic features of R
  - Why use R?
- *Variable
  - Creating variables in R
  - Legal variable name
  - Illegal variable name
- *Data type
- *Advanced Data Structures
  - Vectors
  - Lists
  - Matrices
  - Arrays
  - Factors
  - Data Frames

Module 2 Page No. 2
- *functions
  - creating a function
  - Arguments
  - Nested functions
  - Recursion
- * Vectors
  - Accessing elements
  - Logical Indexing

Module 3 Page No. 3
- *Matrix
  - create a matrix
  - Operation on matrix
- *class
  - Constructing S3 class
  - Constructing S4 class
  - Reference class

Module 4 Page No. 4
- *Different types of Loops
  - Repeat loop
  - While loop
  - For loop
- *Control statements
  - Interruption & Exit loops in R
    - Break statements
    - Next statement
  - If-Else
    - Vectorization with if else
    - ifelse
  - Looping over Non vector Sets
    - The apply family

Module 5 Page No. 5
- *Arithmetic & Boolean Operators & Values
  - R operators
- * Functions in R
  - Writing a function in R
  - Function Call
  - Function Return
  - R Return Values from function
- *Default Values for Arguments
- *Return Value from R Function
  - R function with return value
  - R function without using return
  - R function to return multiple values as a list.

Module 6 Page No. 6
- * Explicit Call Return
  - Deciding whether to explicitly call return
    - No 'return' Keyword
    - With 'return' Keyword
- * Returning Complex objects
  - Create Complex number
- * Functions are objects
- * No Pointers in R
- * Recursion

Module 7 Page No. 7
- * Data Frames
  - Characteristics
- * Making Data frame
  - Create & Name Data Frame
- * Working with Data frame
  - Extract data
  - Expand
  - Add column & Add Row
  - Bind

Module 8 Page No. 8
- *Data Reshaping
  - Joing columns & Rows
- *Melting & Casting of Data
  - reshape2 Package
- *Merging Data Frames

Module 9 Page No. 9
- * Editing & Reading Data from Files
  - Work directory
- * Reading & Writing files
  - Input as csv/R-Excel/R-Binary
  - Read & Analyze
  - Write

Module 10 Page No. 10
- *Exploratory Data Analysis
  - Purpose of EDA
  - Types of Dataset
  - Typology of variables
  - Scale
- *Types of EDA

Module 11 Page No. 11
- *Univariant EDA
  - Measures of Central Tendency
    - Mean
    - Median
    - Mode
  - Measures of Dispersion
    - Min, Max
    - Range
    - Quantile
    - Variance
    - SD, Skewness, kurtosis
- *Bivariant Analysis
  - Crosstabs, covariance, correlation

Module 12 Page No. 12
- * Graphical Methods
  - R Graphics
- * Creating Graphs in R
  - Plotting
    - High-level plotting fns
    - Low-level "
    - Interactive

Module 13 Page No. 13
- * High-level plotting functions
  - Box Plot
  - Bar Plot
  - Pie Chart
  - Histogram
  - Line chart
  - Scatter Plot

Module 14 Page No. 14
- *Low-level Plotting functions
- * Interactive Graphic functions
- * Advanced Graphics
  - Lattice
    - Format
    - Main functions
    - Installing & loading
  - ggplot in R
    - Process of making
    - layers & labels
    - Theme & Facets
- *Visualizing Distribution
- *Saving Graphs

Module 15 Page No. 15
- * Regression Analysis
  - R Linear
  - Multiple Regression
  - Logistic Regression
  - Poisson Regression

MODULE:1 — INTRODUCTION TO R PROGRAMMING

Why use R?

→ Resource for data analysis, data visualization, data science and Machine Learning.

→ Provides many statistical techniques

→ Support different platforms

→ Open-source

→ Large Community Support

→ It has many packages which solve many problems.

Creating Variables in R:

* It does not have command for declaring a variable.

* To assign a value to variable, use ← sign

* To output the variable, just type the variable name.

EXAMPLE:
```
name ← "Sundar"
age ← "40"
```

OUTPUT:
```
name # output "Sundar"
age # output "40"
```

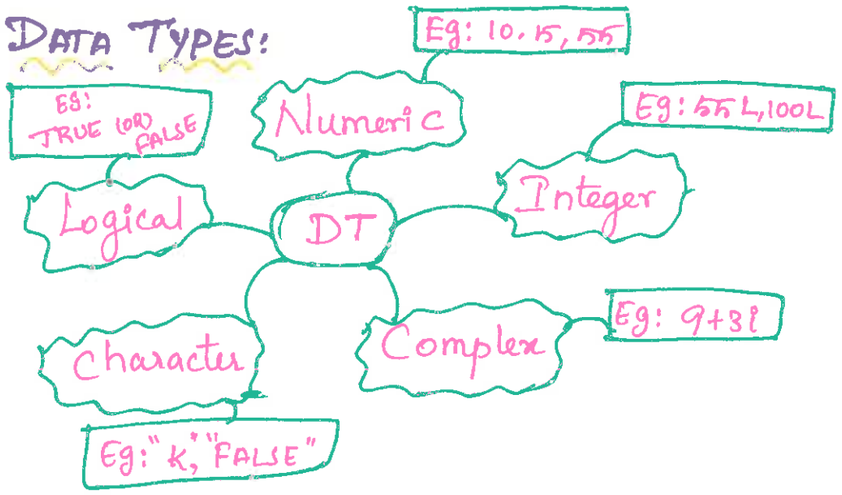
LEGAL VARIABLE NAMES

myVar ← "John"
myVar ← "John"
myvar2 ← "John"
my_var ← "John"
MYVAR ← "John"
.myvar ← "John"

ILLEGAL VARIABLE NAMES

2myvar ← "John"
my-var ← "John"
my var ← "John"
-my_var ← "John"
my_@ar ← "John"
TRUE ← "John"

DATA TYPES:



Advanced Data Structures:

VECTORS → List of items of same type

→ Eg: Fruits, Numbers

```
Fruits ← c("banana", "apple")
numbers ← 1:10
```

LISTS

Collection of different data types which is ordered and changeable

```
thislist ← list("apple", "banana", 1)
```

MATRICES

2D dataset with columns & rows

```
thismatrix ← matrix(c(1,2,3,4,5,6),
              nrow=3, ncol=2)
```

Arrays

Have more than 2D

```
multiarray ← array(thisarray,
             dim=c(4,3,2))
```

FACTORS

Used to Categorize data

```
directions ← factor(c("North", "west"))
```

DATA FRAMES

Data displayed in a format as a table

```
Data_Frame ← data.frame(Training=
  c("strength", "stamina", "other"),
  Pulse=c(100,150,120),
  Duration=c(60,30,45))
```

FUNCTIONS:

- Block of code which runs only when it is called
- Can pass data, known as Parameters into a function
- It can return data as a result.

Creating a function

```
my_function <- function() {
  print("Hello World!")
}
my_function()
```

Arguments

```
my_function <- function(fname) {
  paste(fname, "Griffin")
}
my_function("Peter")
my_function("Lois")
my_function("Stewie")
```

Nested Functions

- Call function with in another function
- Write a function within a function

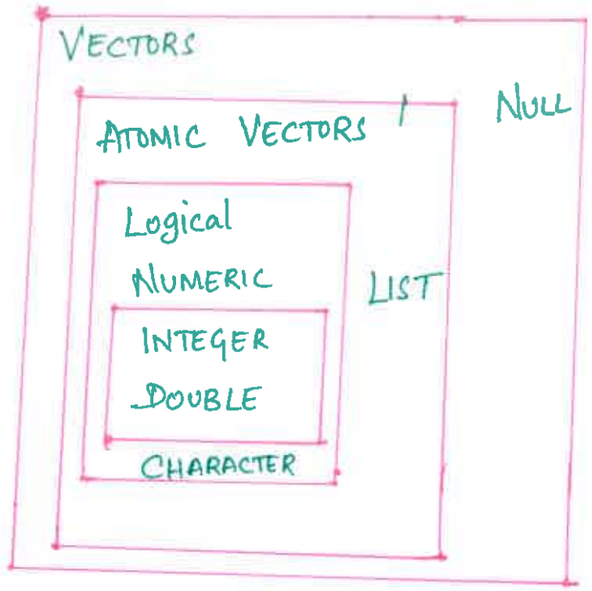
```
* Nested_function <- function(x,y) {
    a <- x+y
    return(a)
  }
  Nested_function(Nested_function(2,2),
  Nested_function(3,3))
```

```
* Outer_func <- function(x) {
  Inner_func <- function(y) {
  a <- x+y
    return(a)
  }
   return(Inner_func)
  }
  output <- Outer_func(3)
  output(5)
```

Recursion : Defined function call itself

```
tri_recursion <- function(k) {
  if(k>0) {
    result <- k + tri_recursion(k-1)
    print(result)
  } else {
    result = 0
    return(result)
  }
}
tri_recursion(6)
```

VECTORS



Accessing elements using position number.

```
X <- c(2,5,8,1,2)
print(X[2])
```

OUTPUT : 5

Accessing Specific values by passing a vector inside another vector.

```
Y <- c(4,5,2,1,7)
print(Y[c(4,1)])
```

OUTPUT : 1,4

Logical Indexing

```
Z <- c(5,2,1,4,4,3)
print(Z[Z>3])
```

OUTPUT : 5,4,4

R PROGRAM TO CREATE A MATRIX:

```
A = matrix (c(1,2,3,4,5,6,7,8,9),
nrow = 3, # No. of rows
ncol = 3, # No. of Columns
# By default matrices are in column wise

byrow = TRUE
)
#Naming Rows : rownames(A)= c("a","b","c")
#Naming Columns: Colnames (A)= c("c","d","e")
Print (A)
```

OUTPUT:

```
  c d e
a 1 2 3
b 4 5 6
c 7 8 9
```

R PROGRAM FOR MATRIX MULTIPLICATION WITH A SCALAR:

```
m<- matrix (1:8, nrow=2)
m<- 2*m
Print(m)
```

OUTPUT:

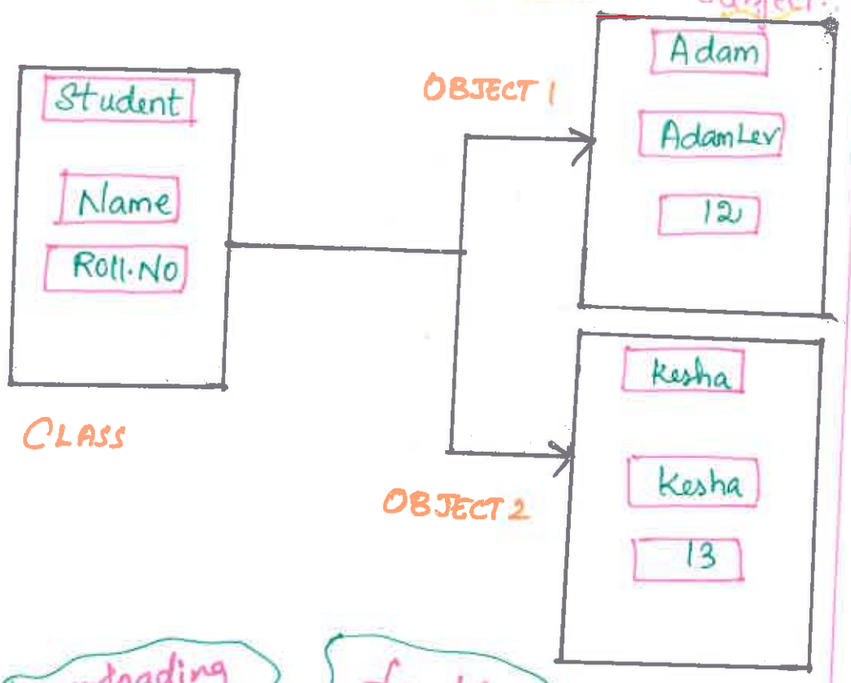
```
2 6 10 14
4 8 12 16
```

R PROGRAM FOR MATRIX MULTIPLICATION:

```
m<- matrix (1:8, nrow=2)
n<- matrix (8:15, nrow=2)
Print (m*n)
```

OUTPUT: 8 80 60 98
18 44 78 120

R-OBJECT ORIENTED PROGRAMING - Class & Subject:





CONSTRUCTING S3 class:

```
a<- list (name = "Adam", Roll_No = 15)
class (a) <- "student"
a
```

OUTPUT:

```
$ name
[1] "Adam"
$ Roll_No
[1] 15
attr(,"class")
[1] "student"
```

CONSTRUCTING S4 class & object:

```
setClass ("student", slots = list (name =
                                   "character",
          Roll_No = "numeric"))

a <- new ("student", name = "Adam",
          Roll_No = 20)
a
```

OUTPUT:

```
Slot "name":
[1] Adam:
Slot "Roll_No":
[1] 20
```

REFERENCE CLASS

Define → setRefClass() function
↳ returns a generator fn

Access → using $ operator.

R Programming Structures

Different types of Loops :

Loop - allows to execute a stmt / group of stmts multiple times

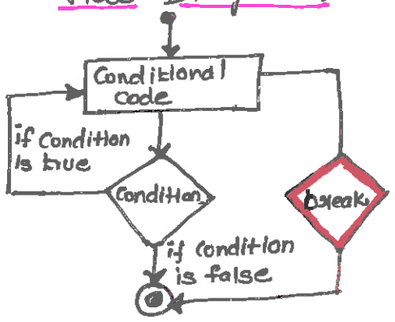
1. Repeat Loop

↳ executes the same code again & again until a stop condition is met.

Basic Syntax

```
repeat {
    Commands
    if (condition) {
      break
    }
```

Flow Diagram



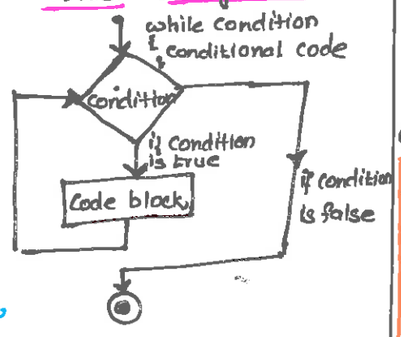
2. While Loop

↳ repeats a stmt / group of stmts while a given condition is true.

Basic Syntax

```
While (test_expression){
    Statement
}
```

Flow Diagram



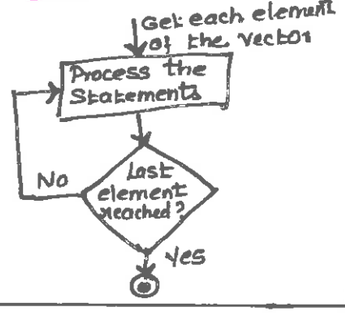
3. For Loop

↳ loop that executes a specific number of times

Syntax

```
for (value in vector){
    Statements
}
```

Flow Diagram



Control Statements : Interruption and Exit Loops in R

a) Break Statement

↳ terminates the loop stmt & transfers execution to immediate LOC.

Usage

* In nested loop - exits from innermost loop
* terminate a case in switch stmt.

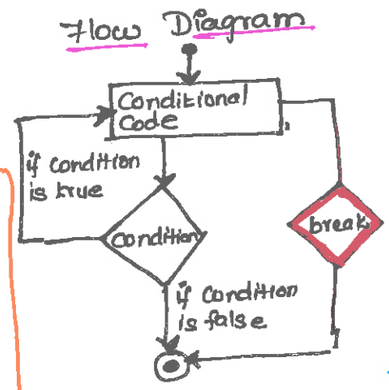
Syntax

```
break
```

Conditional break:

```
for (<value> in <values>) {
    <action part 1>
    if (<condition>) {
        break
    }
    <action part 2>
}
```

Flow Diagram



b) Next Statement

↳ skips the current iteration of a loop without terminating

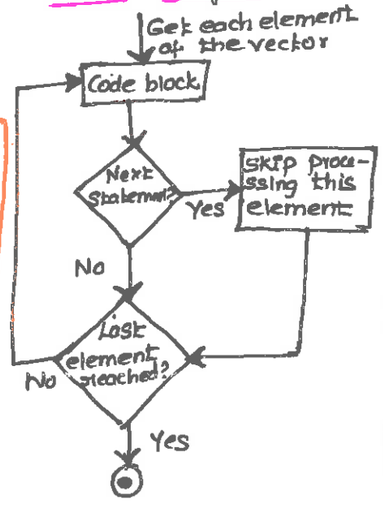
Syntax

```
next
```

Conditional next:

```
for(<value> in <values>) {
    <action part 1>
    if (<condition>) {
        next
    }
    <action part 2>
}
```

Flow Diagram



* jump discontinues a particular iteration & jumps to the next cycle.

Vectorization with if else

If

```
if (condition) {
    #do something
} else {
    #do something else
}
```

ifelse - ifelse(test_expression, x, y)

* the return value is a vector with the same length as test-expression.

Looping Over Non Vector Sets

The apply family

* implement Split-Apply-Combine strategy for data analysis.

* apply() - i/p → Dataframe / matrix
  o/p → vector / list / array

```
apply(X, MARGIN, FUN)
```

X : an array or matrix
Margin: value or range between 1 & 2
FUN : function to apply (mean, median, sum..)

* lapply() - opn's on list objects

```
lapply(x, FUN)
```

X: vector or object
FUN: function applied to each element of x.

* sapply() - i/p - vector / dataframe
  o/p - vector / matrix

```
sapply(x, FUN)
```

* tapply() - Computes a measure (mean, median,..)

```
tapply(X, INDEX, FUN = NULL)
```

X: vector or object
INDEX: list containing factor
FUN : function applied to each element of x.

Arithmetic & Boolean Operators & Values

Operation	Description
x+y	Addition
x-y	Subtraction
x*y	Multiplication
x/y	Division
x^y	Exponentiation
x %% y	Modular arithmetic
x %/% y	Integer Division
x==y	Test of equality
x<=y	Test for less than or equal to
x>=y	Test for greater than or equal to
x && y	Boolean AND for scalars
x || y	Boolean OR for scalars
x & y	Boolean AND for vectors (vector x, y, result)
x | y	Boolean OR for vector (vector x, y, result)
! x	Boolean negation

R Operators

- Arithmetic Operator
- Relational operator
- Logical Operator
- Assignment Operator
- Miscellaneous Operators

Functions in R

↳ objects that take other objects as inputs & return some new object.

↳ set of stmts organized to perform a specific task

Writing a Function in R

```
function_name <- function (arguments)
{
  Function body
}
```

Function Call : `function_name (arguments)`

Function Return

* return values needs to be collected in a seperate variable.

```
rtn_value = function_name (arguments)
```

R Return Values from Function

* functions do processing & return back the result.

`return (expression)`

* value returned from a function can be any valid object.

Default Values for Arguments

* if an argument value is not passed while calling a function, it will take the dfaut value.

Default value
- can be passed only once
- declared while declaring the function

Return Value from R Function

Method 1 : R function with return value

```
function_name <- function (arguments) {
  Statements
  return (value)
}
function_name (values)
```

function_name – name of function
arguments – value passed
return – return a value
function_name (values) – pass value to argument

Method 2 : R function without using Return

```
function_name <- function (arguments) {
  Statements
  value
}
function_name (values)
```

value – return value

Method 3 : R function to return multiple values as a list

```
function_name <- function (arguments) {
  Statements
  return ( list (value 1, value 2, ..., value n))
}
function_name (values)
```

Explicit Call Return

Deciding whether to explicitly Call return :

No 'return' Keyword :- Method 1

Syntax

```
function_name <- function(arguments)
{
    Statement
}
return_variable <- function_name(arguments)
```

* return value collected
* using '<-' operator

Example

```
func_square <- function(x)   #function definition
{
  x * x
}
y <- func_square(5)
y
```

With 'return' Keyword :- Method 2

Example

```
func_square <- function(x)   #function definition
{
  y <- x * x
  return(y)
}

z = func_square(5)   #function call
```

Returning Complex Objects

* Complex function
  - in base package
* Generate Complex number, a+ib
  - call complex function

Example

Create complex number

```
a <- 2
b <- 3
c <- Complex(real=a, imaginary=b)
```

Output:

```
>c
[1] 2+3i
```

Functions are Objects

* all functions like Objects

Print() Function

- Print objects to the screen

```
1>print(1)
[1] >1

1>print("This is R!")
[1] "This is R!"
```

No Pointers in R

Python

```
>>> x=[10, 1, 5]
>>> x.sort()
>>> x
[1, 5, 10]
```

The argument to sort() does not change

R

```
> x <- c(10, 1, 5)
> sort(x)
[1] 1 5 10
> x
[1] 10 1 5
```

```
>vec <- c(1,2)
>v2 <- vec
>v2[2] <- 5
>print(v2)
[1] 1 5
>print(vec)
[1] 1 2
```

"v2 <- vec"

- vec & v2 not entangled to the same object.

Recursion

R Recursive Function

- function calling itself.

Example: Factorial using Recursion in R

```
rec_fac <- function(x) {
    if (x==0 || x==1)
    {
      return(1)
    }
    else
    {
      return(x * rec_fac(x-1))
    }
}
```

output

[1] 120

Example: Sum of Series Using Recursion

Sum = $1^2+2^2+\ldots+N^2$

```
Sum_Series <- function(vec) {
    if (length(vec) <= 1)
    {
      return(vec^2)
    }
    else
    {
      return(vec[1]^2 + Sum_Series(vec[-1]))
    }
}
Series <- c(1:10)
Sum_Series(Series)
```

output

[1] 385

Applications

* Dynamic programming language (DSL) or divide & conquer algorithms.

Recursion Process

- breaking a problem into smaller problems
- built back up piece by piece

Module 7 - Data Frames

Data Frames

- Table (or) 2D array-like structure
- Column contains values of one variable
- Row Contains one set of values from each column.

Used → Store tabular data.

Characteristics

Column
* names - non empty
* Same no. of data items

Row
* names - unique

Data Stored
* numeric, factor (or) Character type

Making Data Frame

Create Data Frame

Using → `data.frame()`
* binds vectors as columns
* names of vectors → column names

Why data Frames?
Because Vectors & matrix can only store element of a single type
Cannot store any real data

```
name <- c("joy", "anne", "chris", "Ben")
age <- c(2,4,6,8)
child <- c(FALSE,TRUE, TRUE, FALSE)
df <- data.frame(name,age,child)
df
```

	name	age	child
1	joy	2	FALSE
2	anne	4	TRUE
3	chris	6	TRUE
4	Ben	8	FALSE

3 Vectors → name, age & child
name → character vector; age & child → integer vector.

Naming Data Frame

Naming columns

```
names(df) <- c("Name", "Age", "child")
```

```
df <- data.frame(Name=name, Age=age, child=child)
```

Data Frame Structure

`str(df)` → name of data frame
displays - internal structure of dataframe in Compact way

```
'data.frame' : 4 obs. of 3 variables
 $ Name : Factor w/ 4 levels ...
 $ Age : num 2 4 6 8
 $ child : logi FALSE TRUE TRUE FALSE
```

`dim(df)` → returns dimension of dataframe
[1] 4 3 → 4 rows, 3 columns

`nrow(df)` [1] 4

`ncol(df)` [1] 3

`Summary(df)` → Statistical Summary & nature of the data.

Working with Data Frame

Extract Data from Data Frame
* extract Specific column using column name

```
result <- data.frame(df$Name, df$Age)
print(result)
```

extracts - 4 rows & first 2 columns

	df.Name	df.Age
1	joy	2
2	anne	4
3	Chris	6
4	Ben	8

```
result <- df[1:2,]
```

	Name	Age	child
1	joy	2	FALSE
2	anne	4	TRUE

→ extracts - first 2 rows & all columns

Expand Data Frame

↳ by adding columns and rows

Add Column

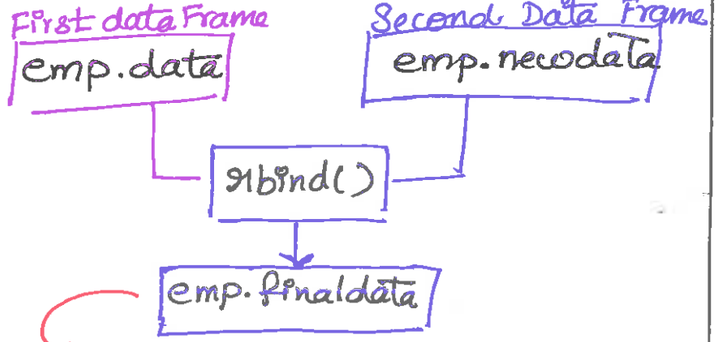
↳ add column vector using a new column name

```
# Create the data Frame
emp.data <- data.frame(
  emp_id = c(1:5),
  emp_name = c("Jes", "Joy", "Anne",
               "Ben", "Jack"),
  Salary = c(5000, 4000, 6000,
             7000, 4500),
  Start_date = as.Date(c("2022-01-01",
    "2022-02-23", "2019-11-15", "2015-01-01")),
  StringsAsFactors = FALSE
)
# Add the "dept" column
emp.data$dept <- c("IT", "HR",
                   "Finance", "IT", "HR")
v <- emp.data
```

↳ by using print(v) → the dataframe is displayed.

Add Row

↳ to add more rows to an existing data frame →
1. Create another data frame with same structure
2. Use the rbind() function




```
emp.finaldata <- rbind(emp.data, emp.newdata)
```

Module 8 - Data Reshaping

Data Reshaping

- Transpose of a Matrix
- Joining Rows & Columns
- Merging of Data Frames
- Melting & casting

Transpose of Matrix

t() function
- i/p → matrix / data frame
- o/p → transpose of matrix / data frame

Syntax: `t(matrix / Data Frame)`

```
first <- matrix(c(1:12), nrow=4, byrow=TRUE)
print("original Matrix")
first                    → prints the matrix

first <- t(first)
print("Transpose of Matrix")
first                    → prints the transpose of the matrix
```

Joining Rows & Columns in Data Frame

↳ join two vectors / merge two data frames using functions

- cbind()
- rbind()

```
cbind(x1, x2, x3)
rbind(x1, x2, x3)
```

x1, x2, x3 → Vectors / matrices / Data Frames

Example

```
name <- c("joy", "anne", "sam", "ben")
age <- c(25, 24, 21, 30)
address <- c("Goa", "chennai", "delhi", "Kerala")
```

#cbind function → combines the vectors into data frame

```
info <- cbind(name, age, address)
```

#creating new data frame

```
newd <- data.frame(name = c("jai", "jes"),
                   age = c("20", "18"),
                   address = c("bihar", "Orissa"))
```

#Rbind function → combine the data frames

```
new.info <- rbind(info, newd)
```

Merging of Data Frames

↳ using merge() function
↳ merges input data frames horizontally

* data frames must have same column names

`merge(dfA, dfB, ...)`

```
d1 <- data.frame(name = c("joy", "anne", "sam"),
                 ID = c("111", "112", "113"))
d2 <- data.frame(name = "jai", "ben")
                 ID = c("114", "115"))
total <- merge(d1, d2, all=TRUE)
```

Melting and Casting

* convert each row into unique id-variable combination & cast it.

2 functions

- melt()
- cast()

```
melt(data, ..., na.rm = FALSE,
     value.name = "value")
```

```
Cast(data, formula, fun.aggregate)
```

- formula → represents the form - data to be reshaped
- fun.aggregate → function aggregate

Example :

	type	year	period	service	incidents
1	A	60	60	127	0
2	A	60	75	63	0
3	A	65	60	1095	3
⋮	⋮	⋮	⋮	⋮	⋮

```
# Melt the Data
molten.ships <- melt(ships, id = c("type", "year"))

#cast Molten Data
recasted.ship <- cast(molten.ships, type + year ~ variable, sum)
```

Tidy Data → Consistently organised data

Tidyr Package

* simplifies process of data reshaping

Install → `Install.packages("tidyr")`

library() → `library(tidyr)`

Functions

- gather() → reshape wide-format to long-format
- spread() → "spread" a key-value pair across multiple columns
- unite() → unite multiple data frame into single column
- separate() → separate data frame column into multiple columns

gather() → `gather(data, key, value, ...)`
- data → data frame name
- key → key column to create
- value → name of value column
- ... → columns to gather from

spread() → `spread(data, key, value)`
- key → column whose values will become variable name
- value → values of new variable

unite() → `unite(data, col, into, sep)`
- col → name of new united column
- into → vector of names for the columns to unite
- sep → Separators

separate() → `separate(data, col, into, sep)`
- col → column to separate
- into → vector of names for column to be separated into
- sep → value to separate the column at

Module 9 Editing and Reading Data from Files

Editing and Reading data from Files

- Read & write into various file formats like csv, excel, xml, etc.

Prereq.: File should be present in current working directory.

Getting & Setting Working Directory

`getwd()` — to check the directory R workspace is pointing to

`setwd()` — set new working directory

Reading & Writing Files

Input as CSV File

CSV File → text file — values in column — separated by comma → filename.CSV → input.csv

#Steps to Read csv file

Create & Save: Create — Notepad; Save as — filename.csv

Read: `read.csv(filename.csv)` → O/P: Dataframe

`read.csv()` → `data <- read.csv("test.csv")`

`is.data.frame()` → `is.data.frame(data)`

Check if dataFrame: `is.data.frame()`, `ncol()`, `nrow()`, `max()`, `subset()` — Analyze

`ncol()` → `ncol(data)`

`nrow()` → `nrow(data)`

`max()` → `Sal <- max(data$Salary)`

`subset()` → `retval <- subset(data, dept=="IT")`

Writing into CSV File

`write.csv()` — create csv file

Step 1: Create a data frame

```
data <- read.csv("test.csv")
retval <- subset(data, as.Date(start_date) >
                 as.Date("2018-01-01"))
```

Step 2: Write filtered data into new file

```
write.csv(retval, "output.csv")
newdata <- read.csv("output.csv")
```

Input as R-Excel file

Packages — XLConnect, xlsx, gdata etc

Install xlsx package → `install.packages("xlsx")`

Verify & load xlsx Package: `any(grepl("xlsx", installed.packages()))`, `library(xlsx)`

* Reading Excel File

`read.xlsx()` → read first worksheet in input.xlsx

```
data <- read.xlsx("input.xlsx",
                  sheetIndex = 1)
```

R-Binary Files → info stored in form of bits & bytes (0's & 1's)

`writeBin()` → `writeBin(object, con)`

`readBin()` → `readBin(con, what, n)`

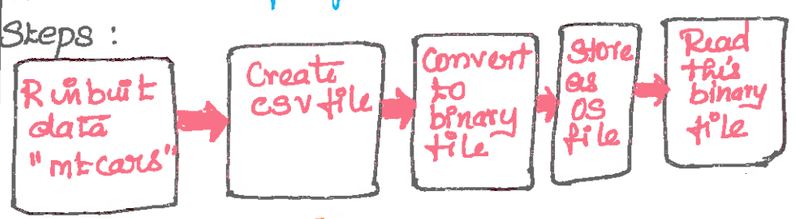
con → connection obj to read/write

object → binary file

what → mode → character, integer etc.

n → no. of bytes to read

Steps:



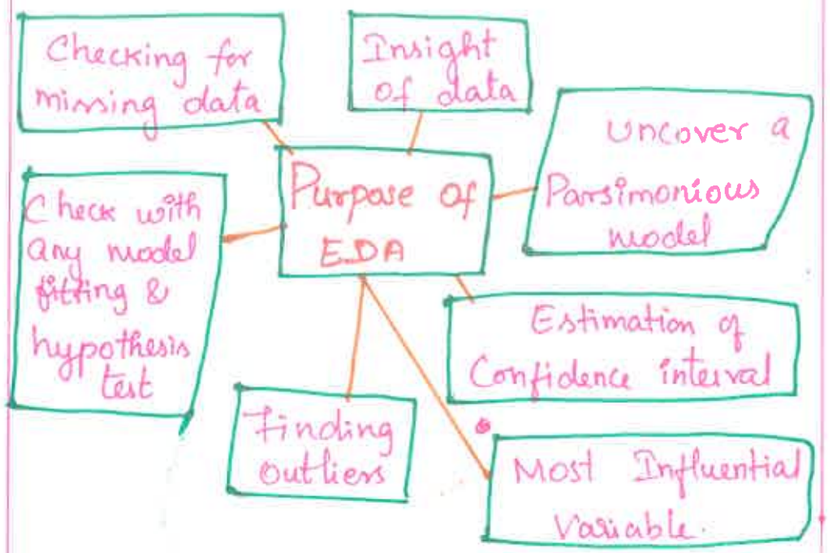
Writing Binary File

```
#Read "mtcars" as csv file & store columns
write.table(mtcars, file = "mtcars.csv",
            row.names = FALSE, na="",
            col.names = TRUE, sep=",")
#Store 5 records from csv file as new data frame
new.mtcars <- read.table("mtcars.csv",
              sep=",", header=TRUE, nrows=5)
#create connection object to write binary
file using mode "wb"
write.filename = file("bin.csv", "wb")
#write column names to connection object
writeBin(colnames(new.mtcars),
         write.filename)
#write records in each column to file
writeBin(c(new.mtcars$cyl,
           new.mtcars$am,
           new.mtcars$gear),
         write.filename)
#close
close(write.filename)
```

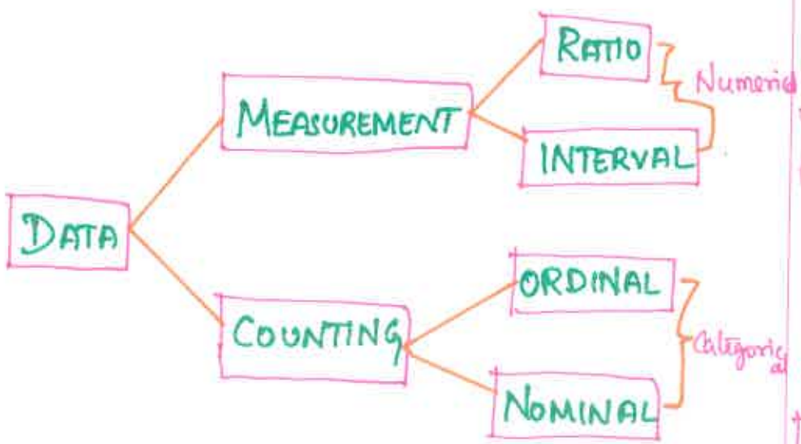
Reading Binary File

```
#create connection object — mode "rb"
read.filename <- file("web/com/binmtcars.dat",
                      "rb")
# Read column names. n=3
column.names <- readBin(read.filename,
                character(), n=3)
#Read column values. n=18
bindata <- readBin(read.filename, integer(),
                   n=18)
# Read value — 4th byte to 8th byte — "cyl"
cyldata = bindata[4:8]
# Read values — 9th — 13th byte — "am"
amdata = bindata[9:13]
# Read values — 14th — 18th byte — "gear"
geardata = bindata[14:18]
# Combine all read values to data frame
finaldata = cbind(cyldata, amdata,
                  geardata)
colnames(finaldata) = column.names
```

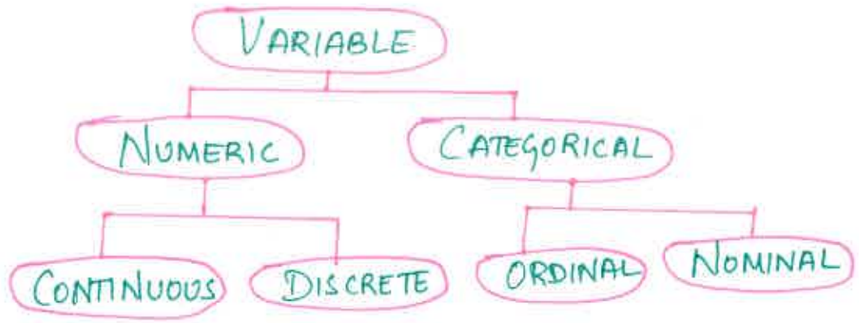
EDA – Summarizing and visualizing a dataset.



TYPES OF DATASET:



TYPOLOGY OF VARIABLES:



SCALE:

NOMINAL – Used to label variables without any quantitative value.

EG: MALE , FEMALE , COLOR

ORDINAL – To remember the ordinal sounds like order.

EG: Rank Order of winners – First, second, Third...

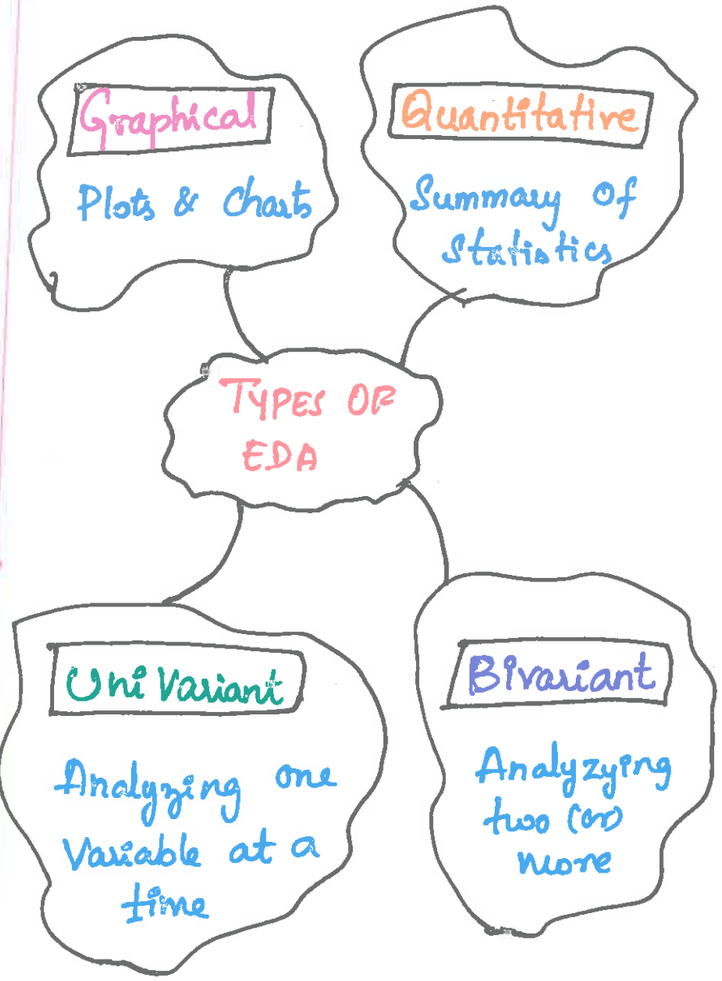
INTERVAL – Concerned about both order & difference between variables.

EG: Performance rating on scale 0-10, 8.2, 9.1, 9.6...

RATIO – It tells about order of variables, difference between them, & absolute Zero.

EG: Time to finish in seconds. 15.2, 14.1, 13.4...

TYPES OF EDA:



EDA also depends on role of variable being examined:

* Outcome Variable
* Exploratory Variable

UNIVARIANT EDA

Measures Of Central Tendency:

- MEAN
- MEDIAN
- MODE

MEAN

- ARITHMETIC MEAN [AVERAGE]
  - Sum / Size = $\frac{a+b+c+\dots+n}{n}$
  - Used for most situations
- GEOMETRIC MEAN [AVERAGE FACTOR]
  - $\sqrt[n]{abc\dots n}$
  - Investments, growth, area, volume
- HARMONIC MEAN [AVERAGE RATE]
  - $\frac{n}{\frac{1}{a}+\frac{1}{b}+\frac{1}{c}+\dots+\frac{1}{n}}$
  - Speed; Production, cost.

MEDIAN

- Middle of sorted list
- Wildly Varying Samples

MODE

- Most Popular Value
- No Compromises [Winner takes all]

MEASURES OF DISPERSION

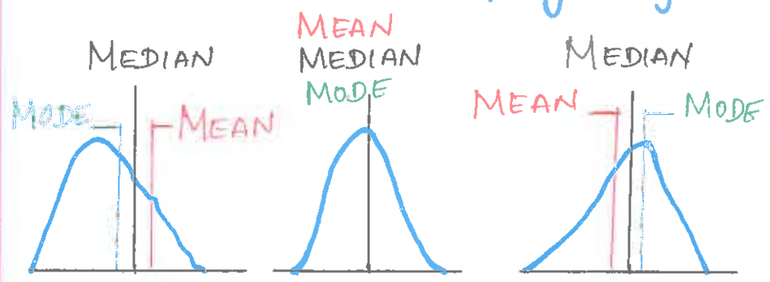
- MIN, MAX — Min & Max of given variable.
- RANGE — Largest Value − Smallest Value
  - Measure of how far apart the entire data spreads in value.
- QUARTILE
  - First Quartile — 25% of data
  - Second Quartile — 50% of data
  - Third Quartile — 75% of data
- VARIANCE
  - Sample Variance — How the data value dispersed around the mean

$$S^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i-\bar{x})^2$$

  - Population Variance — Defined interms of population mean & size

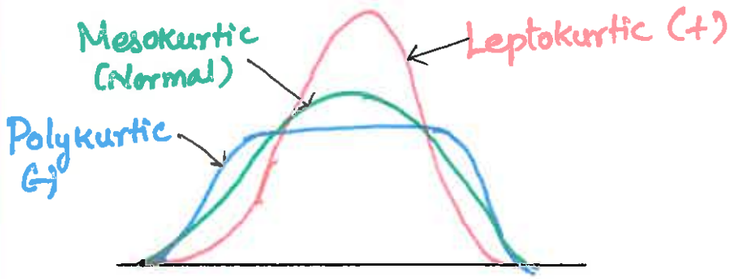
$$\sigma^2 = \frac{1}{N}\sum_{i=1}^{N}(x_i-\mu)^2$$

- STANDARD DEVIATION — $\sqrt{\text{Variance}}$
- SKEWNESS — Measure of Symmetry



$$\bar{\mu}_3 = \frac{\sum_i^N (X_i-\bar{X})^3}{(N-1)*\sigma^3}$$

- KURTOSIS — Measure of 'tailedness' of PDF.



BIVARIANT ANALYSIS

- COVARIANCE
  - Sample
    - $S_{xy} = \frac{1}{n-1}\sum_{i=1}^{n}(x_i-\bar{x})(y_i-\bar{y})$
  - Population
    - $\sigma_{xy} = \frac{1}{N}\sum_{i=1}^{N}(x_i-\mu_x)(y_i-\mu_y)$
- CORRELATION COEFFICIENT
  - Sample
    - $r_{xy} = \frac{S_{xy}}{S_x S_y}$
  - Population
    - $\rho_{xy} = \frac{\sigma_{xy}}{\sigma_x \sigma_y}$
- CROSS TABS
  - Summarizing a data from one (or more source into a consise format

Module 12 R Graphics

Graphical Methods:

R Graphics

Traditional graphics	Grid graphics
* Available in R from the beginning	* Recent
* Rich collection of tools	* Low level tool
* Not very flexible	* Flexible

Creating graphics in R:

For plotting

plot(), points(), lines(), text(), mtext(), axis(), identify(), etc...

Plotting Commands:

* High-level plotting functions
* Low-level plotting functions
* Interactive graphics functions

☒ R maintains a list of graphical parameters which can be manipulated to customize a plot.

The R function plot() - generic func
* dependent on the class of the first argument.

```
plot(y~x)
        # use a formula to
          specify the graph
plot(x,y)
        # x and y must be
          same length
```

Ex:

```
plot((0:20)*pi/10, sin((0:20)*pi/10))
plot((1:30)*0.92, sin((1:30)*0.92))
```

Plot of Vectors

```
# One vector x (plots the vector
    against the index vector)
x ← 1:10
plot(x)
# Scatter plot of two vectors
  x and y.
x ← -30:30
y ← 3*x+2+rnorm(length(x),
                sd=20)
```

plot(x,y)

Plot of data frame elements:

Ex:

```
> airquality[1:2, ]
```

Ozone	Solar	Wind	Temp	day
141	190	7.4	67	1
236	118	8.0	72	2

```
> plot(airquality)
```

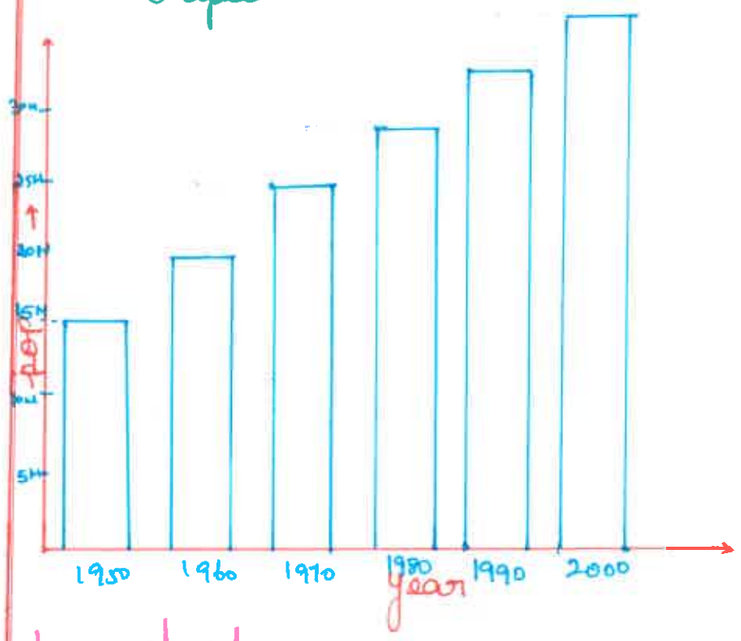
Multiple plots in the same window

```
> par(mfrow = c(2,1))
> plot(airquality$Ozone, airquality
  $Temp, main = "airquality $Ozone,
   airquality $Temp")
> attach(airquality)
> plot(Ozone, Temp, main) =
    "plot(Ozone, Temp)")
> detach(airquality)
```

Module 13 - High Level Plotting Functions

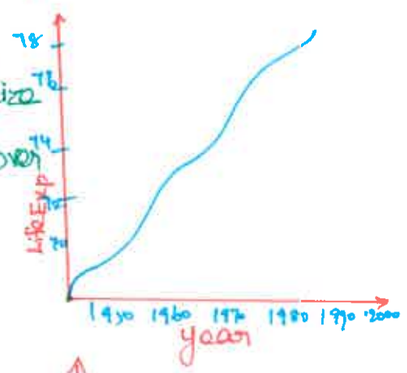
Bargraph :

* Categorical data with rectangle-shaped bars.



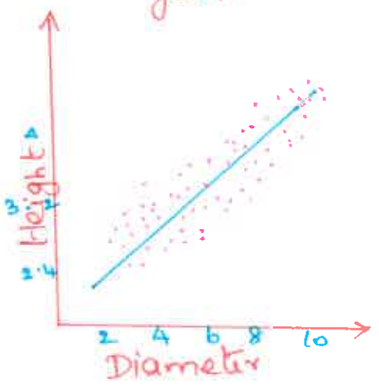
Line chart :

* used to visualize a trend in data over intervals of time

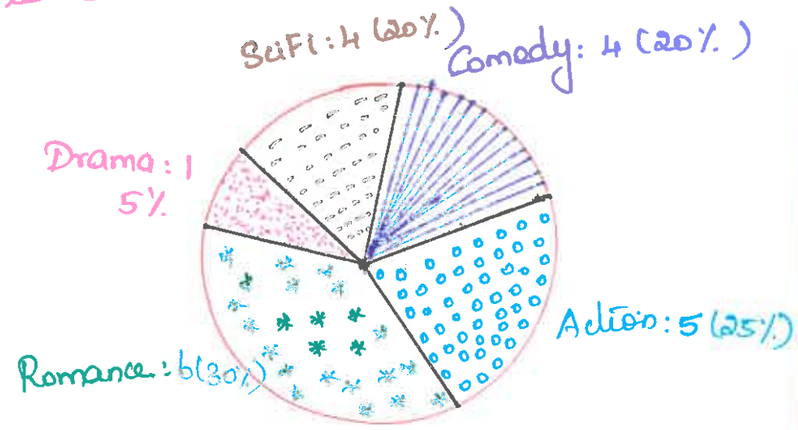


Scatter plot :

* Identification of correlation relation-ship between two numerical variables.



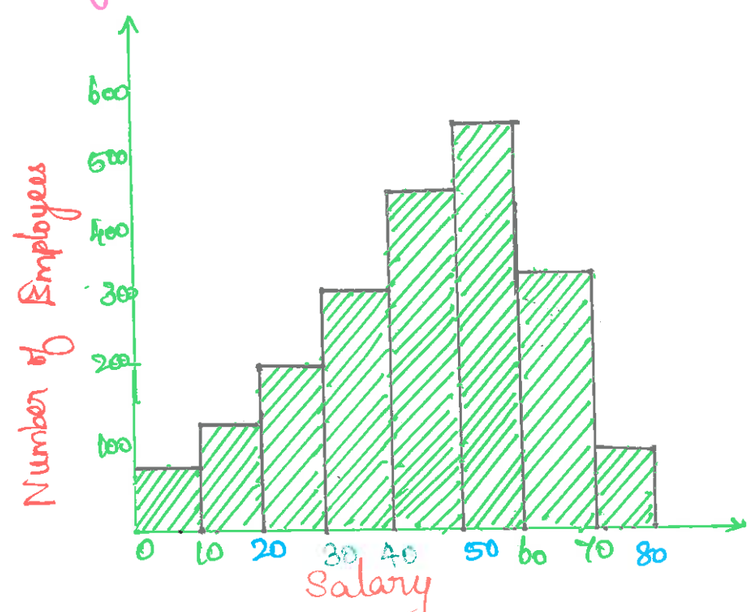
Pie chart



Favorite type of movie

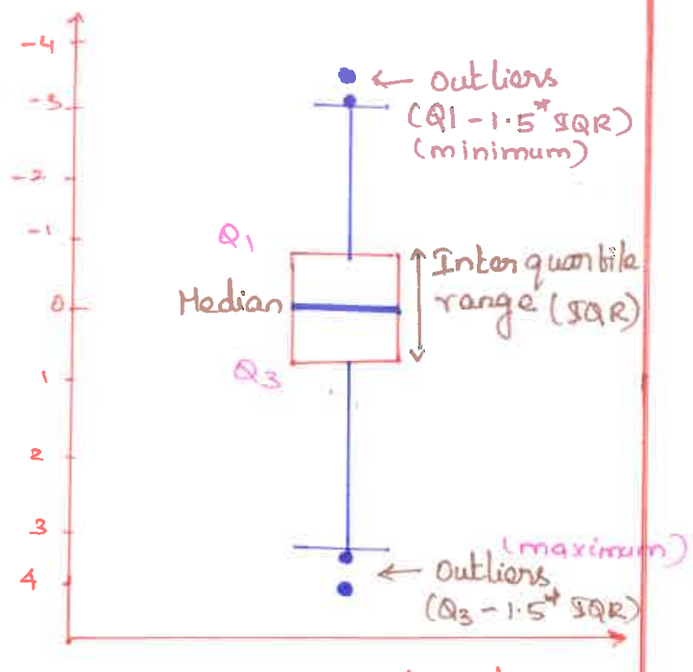
* Circular statistical graph

Histogram :



* Used to illustrate the major features of the distribution of the data.
* Summarize discrete or continuous data.

Box plot :



Different parts of boxplot

Minimum → minimum value in the given dataset

First Quartile (Q_1) → median of the lower half of the dataset

Median → Middle value of the dataset

Third Quartile (Q_3) → median of the upper half of the data.

Maximum → maximum value of dataset

Interquartile range (IQR) → $IQR = Q_3 - Q_1$

Outlier → Outliers are $< Q_1 - (1.5 \cdot IQR)$ or $> Q_3 + (1.5 \cdot IQR)$

Low-level plotting functions

*add extra information

points(x,y) / lines(x,y) → add points / connected lines to current plot

```
points(speed[speed<15], dist[speed<15], pch="S", col="blue")
lines(lowess(cars), col="red")
```

text(x,y, labels,...)

```
text(1:5, rep(6,5), labels=c(0:4), cex=1:5, col=1:5)
```

abline(a,b) abline(h=y) abline(v=x)

```
abline(a=NULL, b=NULL, h=NULL, v=NULL, reg=NULL, coef=NULL, untf=FALSE, ...)
```

a → intercept & slope object; reg → regression; untf → untransform

polygon(x,y,...)

legend(x,y, legend,...)

title(main, sub)

axis(side,...)

Interactive Graphics function.

- locator() → reads position of graphics cursor
- identify() → reads position of graphics pointer

locator(n=512, type="n", ...)

* locator → support screen devices such as 'X11', 'windows' & 'quartz'.

identify(x, y=NULL, labels=seq_along(x), pos=FALSE, n=length(x), plot=TRUE, ...)

↳ generic function

Advanced Graphics

Lattice Graphs

*display multivariate relationships

Main functions

xyplot()	splom()
cloud()	stripplot()
bwplot()	dotplot()
barchart()	histogram()
densityplot()	qq(math)
contourplot()	levelplot()
parallel()	wireframe()

* install & load

```
install.packages("lattice")
library("lattice")
```

```
histogram(~vot)        stripplot(vot~phon, data=df)
dotplot(vot~stress, data=df)
xyplot(y~x|f)
```

ggplot2

*data visualization package

1. Setup
   ↳ ggplot(df)
2. Layers – 'geoms'
   ↳ geom_point()
   ↳ geom_smooth()
3. Labels – 'labs'
4. Theme
   *adjusting size of labels
   ↳ text()
   ↳ blank()
   ↳ discrete()
5. Facets – facet_wrap()

```
library(ggplot2)                          → basic plot
qplot(phon, vot, data=df)
qplot(phon, vot, data=df, color=stress)
qplot(vot, data=df, geom="density", color=phon)
qplot(phon, vot, data=df, facets=.~stress)
g <- ggplot(df, aes(phon, vot))
g + geom_point()
```

save plots → pdf(file="fileName.pdf") dev.off()

Visualizing Distribution

Visualise distribution of a variable → Categorical / Continuous

Categorical → small set of values
↳ variables saved as factors / character vectors
↳ bar chart

Continuous → infinite set of ordered values
↳ eg:- numbers & date-times
↳ histogram

*tall bars – common values of variable
Shorter bars – less-common values

Saving Graphs

↳ plot exported from menu in plot panel.

Plots panel → Export → Save as Image or Save as PDF

1. Specify files → using function
   + jpeg(), png(), svg() or pdf()
   height & width of image → can be used
2. Create plot
3. close file → dev.off()

```
jpeg("c:/mygraphs/myplot.jpg")
plot(x)
dev.off()
```

example

Regression Analysis

* establish relationship model between
  - predictor variable → gathered through experiments
  - & response variable → derived from predictor variable

Linear Regression

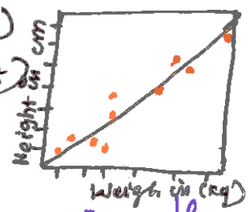
→ relationship estimated between two variables.
→ produces straight line

$y = ax + b$
- x → predictor (or) independent variable
- y → response (or) dependent variable
- a, b → co-efficients

lm(formula) → creates linear regression model

Steps:- * Gather sample height & weight

```
x ← c(151, 174, 128, 163, 152, 131)
y ← c(60, 80, 50, 65, 55, 58)
```



* Apply lm() function

```
relation ← lm(y ~ x)
```

* Create mathematical equation using the co-efficients.
* Obtain residuals → average error in Prediction
* Use predict() function → predict the weight of new persons

predict(object, newdata)

```
# Weight of person with height 170.
a ← data.frame(x = 170)
result ← predict(relation, a)
```

Multiple Regression

* uses more than one predictor variables

$y = a + x_1 * b_1 + x_2 * b_2 + \dots + x_n * b_n$

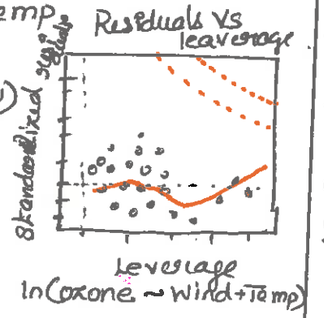
- y – response variable
- a, b1, b2, ... bn — co-efficients
- x1, x2, ... xn – predictor variables

`lm(y ~ x1+x2+x3..., data)`

- formula → relation b/w response & predictor vble
- Vector on which formula is applied

```
input ← airquality[1:50,
        c("Ozone", "Wind", "Temp")]
model ← lm(Ozone ~ Wind + Temp,
        data = input)
png(file = "multiple Reg.png")
plot(model)
dev.off() # save file
```

→ # O/p in PNG file



Logistic Regression

* predicts value with a range – for categorical data.

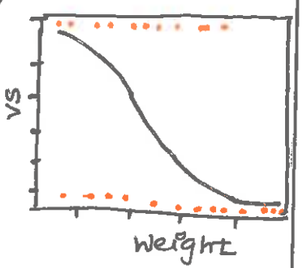
$y = \frac{1}{1 + e^{-z}}$

y – response variable
z → eqn. of independent variables (or) features

`glm(formula, data, family)`
- data → dataframe
- family → type of fn – "binomial"

```
#create logistic model
model ← glm(formula = vs ~ wt,
            family  = binomial,
            data    = mtcars)
```

```
# creating a range of wt values
x ← seq(min(mtcars$wt),
        max(mtcars$wt),
        0.01)
# predict using weight
y ← predict(model,
        list(wt = x),
        type = "response")
png(file = "logReg.png")
plot(mtcars$wt, mtcars$vs, pch = 16,
     xlab = "weight", ylab = "VS")
lines(x, y)
dev.off()
```



Poisson Regression

* model count variables

$\log(y) = a + b_1 x_1 + b_2 x_2 + b_n x_n \dots$

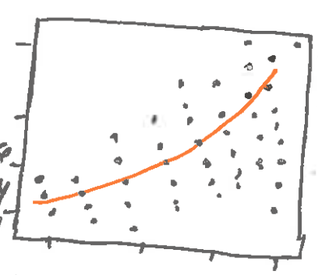
y – response variable
a & b – numeric co-efficients
x – predictor variable

`glm(formula, data, family)`
- data → dataset
- family → specify details of model – "poisson"

```
output ← glm(formula = breaks ~ wool + tension,
             data = warpbreaks,
             family = poisson)
```

Visualizing:-

* jtools
  - plot_summs() } visualize summary of model
  - plot_coefs() }
  - → allows to compare different models with ggplot2



SIMATS

SCHOOL OF ENGINEERING

Engineer to Excel

Approved by AICTE | IET-UK Accreditation

Saveetha Nagar, Thandalam, Chennai - 602 105, TamilNadu, India